Four harnesses, one loop.

The five ideas from the webinar as keys, commands and settings in four tools. Front: how. Back: what differs.

CC Claude Code · GH Copilot CLI · OC OpenCode v2 · CX Codex CLI

docs checked 7 Nov 2026 · nextacademy.io



Getting started · Start

table: Picking a tool

Four tools, one terminal. The first five things your hands need.

	CC Claude Code	GH Copilot CLI	OC OpenCode	CX Codex CLI
Start in a project	`claude`	`copilot`	`opencode`	`codex`
Cancel the turn	`Esc`	`Esc Esc`	`Esc`	◌ not in the docs Ctrl+C quits
Switch mode	`Shift+Tab`	`Shift+Tab`	`Shift+Tab` cycles agents	`/permissions`
Resume a session	`claude -c`	`--continue`	`run --continue`	`codex resume`
Help	`/help`	`/help`	`Ctrl+P` command palette	`?` shortcut list
Settings file	`settings.json` .claude/ · ~/.claude/	`settings.json` .github/copilot/	`opencode.json` ~/.config/opencode/	`config.toml` .codex/ · ~/.codex/



The loop · Step 1

table: The loop

The model chooses, the harness runs the tool, the result comes back. You steer it.

Side question	`/btw`	`/btw` alias /ask	`session.aside` no default key	`/side` alias /btw
Mention a file	`@`	`@`	◌ not in the docs	`@`
Shell, not the model	`!` line prefix	`!` line prefix	◌ not in the docs	`!` line prefix
Compact the context	`/compact`	`/compact`	`Ctrl+X c`	`/compact`
Fresh start	`/clear`	`/clear`	`Ctrl+X n`	`/clear`
Reasoning effort	`/effort high`	`--reasoning-effort`	`Ctrl+T` cycles model variants	`/model` sets the effort too

Practice in the course: `Task 03`, auth and clashes. Watching the loop on a real codebase.



The gate · Step 2

table: The gate

How much it may do without asking, and what it may never do.

Show or set rules	`/permissions`	`/permissions`	◌ not in the docs in opencode.json	`/permissions`
Plan before acting	`/plan`	`/plan`	`plan agent`	`/plan`
Ask less	`acceptEdits`	`assisted`	`run --auto` all but deny	`approve-for-me` auto_review
Deny a path	`deny rule`	`--deny-tool` exact paths	`effect: deny`	`profile glob` not promised

Lock the tests — deny edits on test files

- CC `"deny": ["Edit(**/*.test.ts)"]` in .claude/settings.json; also stops sed, tee and redirects
- GH `--deny-tool='write(src/foo.test.ts)'` exact path; globs only in managed settings
- OC `{ "action": "edit", "resource": "*.test.ts", "effect": "deny" }` in opencode.json
- CX `[permissions.<name>.filesystem]` path globs in a profile; blocking edits is not promised

Bypass (`--dangerously-skip-permissions`, `--yolo`): the Claude Code docs advise containers and VMs only.



Advice and law · Step 3

table: Advice and law

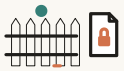
An instruction file is advice the model may ignore. A hook that blocks is law.

	CC Claude Code	GH Copilot CLI	OC OpenCode	CX Codex CLI
Advice file	`CLAUDE.md` else AGENTS.md	`AGENTS.md` and its own file	`AGENTS.md` no CLAUDE.md	`AGENTS.md` fallbacks via config
Hook file	`settings.json` .claude/	`.github/hooks/` NAME.json	`.opencode/plugins/` a TS plugin	`hooks.json` .codex/
Before-tool event	`PreToolUse`	`preToolUse`	`execute.before`	`PreToolUse`
Blocks with	`exit 2`	`exit 2`	`effect = deny`	`exit 2`

A hook that blocks — where it lives, what it does

- CC `.claude/settings.json → "hooks" → PreToolUse → the command exits 2 → the tool does not run`
- GH `.github/hooks/NAME.json → preToolUse → exit 2, or permissionDecision: "deny" → denied`
- OC `.opencode/plugins/x.ts → ctx.permission.hook("evaluate") → event.effect = "deny"`
- CX `.codex/hooks.json → PreToolUse → exit 2 with the reason on stderr → the tool does not run`

Practice in the course: `Task 06`, context and CLAUDE.md. Write the rule, then enforce it with a hook.



The fence · Step 4

table: The fence

An OS sandbox limits what a shell command can touch, whatever the model decides.

Turn it on	`/sandbox` off by default	`/sandbox enable` off by default	○ no sandbox shell runs as you	`on by default` sandbox_mode
Runs on	`Mac Linux WSL2` not native Windows	`Mac Linux Win11`	⊘ —	`every OS` native on Windows

Close the escape hatch — the unsandboxed retry

- CC `"sandbox": { "enabled": true, "allowUnsandboxedCommands": false }` Windows: run inside WSL2
- GH `/sandbox enable` · `"sandbox.allowBypass": false` off by default; managed settings can enforce both
- OC no sandbox: put a container or VM around OpenCode, policies for hard limits
- CX `sandbox_mode = "workspace-write"` leaving asks, or auto review decides; rules can forbid prefixes



Letting go · Step 5

table: Letting go

Autonomy is earned by verification. Put the boundary in the goal: tests pass, tests unchanged.

Run to a goal	`ALL FOUR /goal <objective>` · Copilot: alias of /autopilot			
Who checks the goal	`small model` reads chat, no tests	`the model` decides it is done	`a reviewer` criteria not in docs	`same chat`
Fan out to PRs	`/batch` a PR per unit	`/fleet` no PRs	⊘ — subagents only	⊘ — cloud tasks

Nobody watching — the headless call, also in CI

- CC `claude -p "<prompt>"` a Stop hook can run the tests and exit 2 · claude-code-action@v1
- GH `copilot -p "<prompt>" --allow-tool=…` --no-ask-user for scripts · in a run step, no action
- OC `opencode run "<prompt>" --auto` auto-approves all but explicit deny · in a run step; an action is not in the docs
- CX `codex exec "<prompt>"` scripted runs without human interaction · codex-action@v1

Practice in the course: `Task 15`, letting go.

● has it ◍ partly, or works differently ⊘ not available ◌ not in the docs: we looked, found nothing

Tools change fast; on screen, cells with a docs page link to it. Adam Furmanczuk · nextacademy.io → turn over

Mastering Claude Code

- **Course** 24–26 November, online · 25 % off within 7 days of the session
- **Repo** Verification kit, a course task, this sheet: {{GIFT_URL}}

QR

The session, compared.

What each tool gives you for every idea, and where it falls short. A Claude Code gap names the workaround we use.

● has it ◐ partly, or works differently ○ not available ◌ not in the docs

docs checked 7 Nov 2026 · nextacademy.io

Mechanism	CC Claude Code	GH GitHub Copilot CLI	OC OpenCode v2	CX Codex CLI
• The loop				front: Step 1
Read, act, verify the agent loop	● Gather context, take action, verify results. The phases blend.	● VS Code calls it the agent loop: understand, act, validate.	● Model steps with tool calls; the step limit is configurable. `steps`	● Plan, use tools, check, adapt as needed.
• The gate				front: Step 2
Permission modes how much it may do without asking	● Six modes; deny rules hold in every one. `Shift+Tab`	● default, assisted, allow-all; plus plan and autopilot.	◐ Rules are allow, ask or deny. A plan agent; autoaccept as a switch.	● Built differently: a sandbox mode times an approval policy.
Lock the tests deny edits by path	● A deny rule covers every file tool, also sed, tee and redirects.	◐ Exact paths only on the CLI; globs only in managed settings.	● An ordered list; the last matching rule wins.	◐ Path globs in a profile; blocking edits is not promised.
• Advice and law				front: Step 3
Hooks that block code runs before the tool does	● PreToolUse blocks with exit code 2.	● preToolUse can deny, also with exit 2. postToolUse cannot block.	● Through a plugin: set the permission decision to deny.	● PreToolUse blocks with exit 2. Docs: not a complete enforcement boundary.
The instruction file advice, and one file for every tool	◐ Context, not enforced. AGENTS.md is read only without a CLAUDE.md. **Workaround:** Put @AGENTS.md inside CLAUDE.md.	● Combines its own file with AGENTS.md, CLAUDE.md and GEMINI.md.	◐ AGENTS.md only; v2 does not use CLAUDE.md as a fallback.	◐ AGENTS.md from the root down; CLAUDE.md only as a configured fallback.
Law configuration the harness enforces	● Deny holds in every mode; managed settings override all.	◐ Managed settings are a baseline below user and repo; some keys, such as bypass and sandbox, are enforced.	◐ Project config can be overridden; policies in the system config are hard.	● Read for trusted projects; admins enforce requirements.toml.
• The fence				front: Step 4
Sandbox, no escape hatch OS-level limits for shell commands	◐ Off by default. macOS, Linux, WSL2; a second key turns off the unsandboxed retry. Not on native Windows. **Workaround:** On Windows, run Claude Code inside WSL2 or a container.	◐ macOS, Linux, Windows 11 25H2+; off by default. sandbox.allowBypass: false closes the hatch.	⊘ The shell runs with the host user's full authority.	◐ Every OS, natively on Windows. Leaving asks, or auto review decides; rules can forbid prefixes.
• Letting go				front: Step 5
Run to a goal keep going until a check passes	◐ A small model judges after each turn. It reads the chat and runs no tests. **Workaround:** A command Stop hook runs the tests and exits 2 while they fail.	◐ Alias of autopilot; the model decides it is done.	◐ A reviewer confirms the objective (v2.2.1); what it checks is not in the docs.	◐ The same chat decides. Put the tests in the goal.
Many PRs fan out, and issue to PR	◐ /batch: 5 to 30 units, a worktree and a PR each. Issue to PR only via the Action. **Workaround:** The GitHub Action turns an @claude comment into a PR.	◐ /fleet runs in parallel without PRs; assign an issue and the cloud agent opens one.	○ No fan-out, only subagents. Issue to PR is not in the docs.	◐ No /batch. @codex in a comment starts a cloud task; the docs call it legacy.
Nobody watching headless and CI	● Headless, a GitHub Action, scheduled tasks.	● Headless in a run step; no official action.	● Headless in a run step; an action is not in the docs.	● codex exec for scripted runs without human interaction; a GitHub action.
• Picking a tool				front: Getting started
Model and licence bring your own model; read the harness	◐ Officially Claude only; other models via a compatible endpoint work in our tests, unsupported by Anthropic. Proprietary. `ANTHROPIC_BASE_URL`	◐ Bring your own key, also a local endpoint. Proprietary. `COPILOT_PROVIDER_BASE_URL`	● A provider catalog; finds Ollama, LM Studio and vLLM locally. MIT.	● Local open source models. Apache-2.0. `codex --oss`
Best of N several attempts, keep the best	⊘ Not built in. **Workaround:** One --worktree or --cloud session per attempt, then compare.	◌ Not in the docs.	◌ Not in the docs.	● A cloud task runs up to four attempts (best-of-N). `codex cloud exec --attempts 4`

Read before you copy. An `Edit` deny stops file tools and recognised shell writes, not a script that opens the file; the sandbox closes that gap. "Not in the docs" means we searched the official docs and found nothing. On screen, cells with a docs page link to it.

From the webinar "Mastering Claude Code"
Adam Furmanczuk · nextacademy.io
Front: how. Back: what differs.